

The background image shows the interior of a ship's bridge at night. The room is dimly lit, with a warm, reddish-orange glow from the control panels and monitors. In the center, there is a large steering wheel. Surrounding it are several electronic displays and control panels. On the left, there are two large monitors showing radar or sonar data. In the center, there is a smaller monitor displaying a ship's layout or navigation information. On the right, there are more monitors and control panels. Above the main control area, there is a row of seven circular gauges or indicators, each with a different colored face (blue, yellow, red, blue, yellow, yellow, blue). The windows in the background show a dark blue sky with some clouds. The overall atmosphere is one of a modern, high-tech maritime control environment.

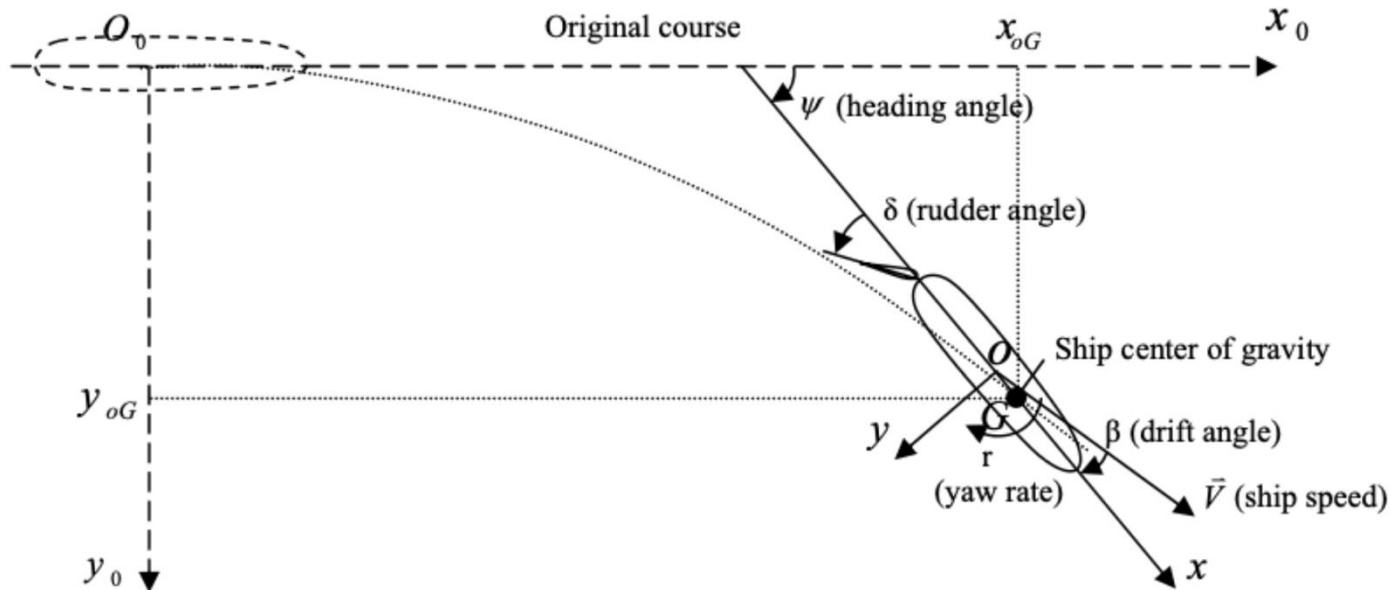
Ship Motion Autonomous Control in 3-DoF

Martina De Angelis (r1125657), Seppe Hermans (r0939189)

Mathematical Model

Earth-fixed coordinate system: $O_0 - x_0 y_0 z_0$

Body-fixed coordinate system: $O - x y z$



Ship speed: V

Speed in the direction of the body-fixed axis: u, v

Heading angle: ψ

Yaw rate: $r = \dot{\psi}$

Drift angle: β (between V and the x -axis)

Mathematical Model

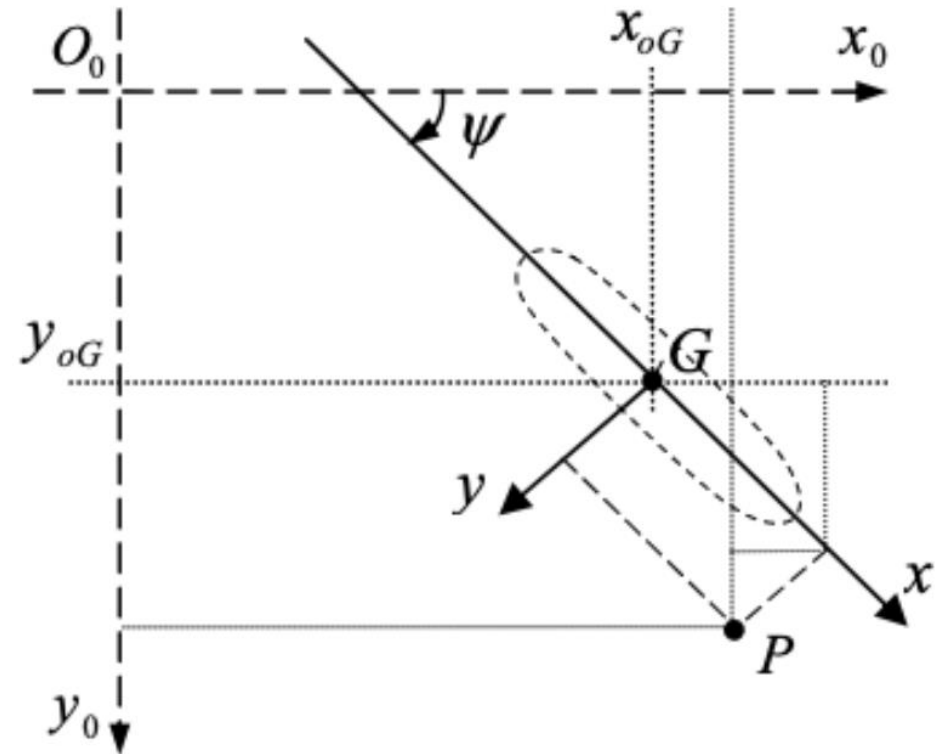
Denoting:

$$\begin{aligned}x'_0 &= x_0 - x_{oG} \\y'_0 &= y_0 - y_{oG} \\z'_0 &= z_0\end{aligned}$$

We have:

$$\begin{aligned}x'_0 &= x \cos(\psi) - y \sin(\psi) \\y'_0 &= x \sin(\psi) + y \cos(\psi) \\z'_0 &= z\end{aligned}$$

```
R = np.array([
    [np.cos(psi), -np.sin(psi), 0],
    [np.sin(psi),  np.cos(psi), 0],
    [0,           0,           1]
])
```



The Fossen Model

$$M \dot{v} + C(v)v + D(v)v + g(\eta) + g_o = \tau + \tau_{wind} + \tau_{wave}$$

M: system inertia matrix

C: Coriolis-centripetal matrix

D: damping matrix

g: vector of gravitational/buoyancy forces

g_o: vector used for pretrimming

τ : vector of control inputs

τ_{wind} : vector of wind forces

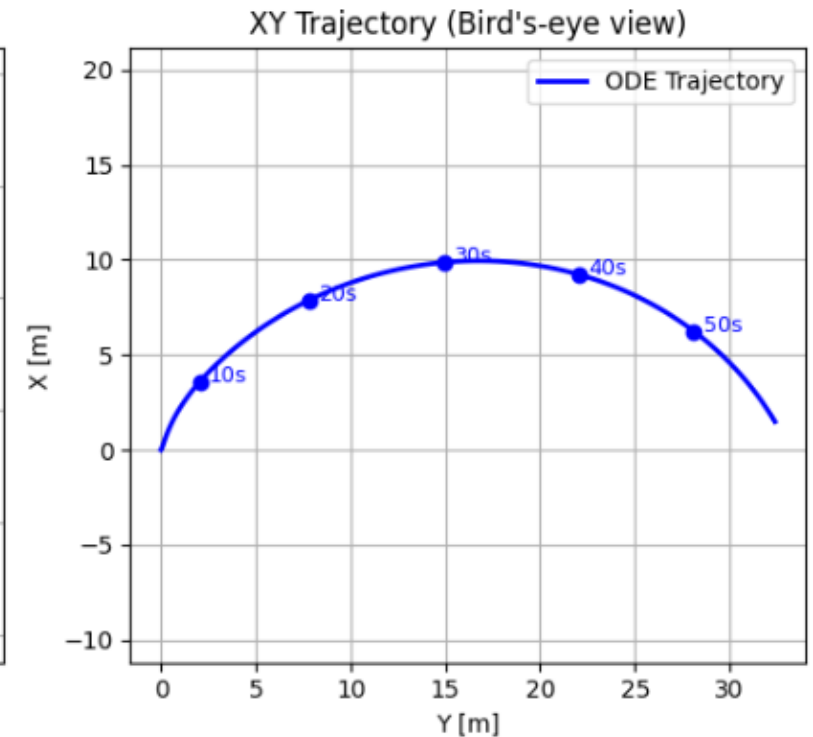
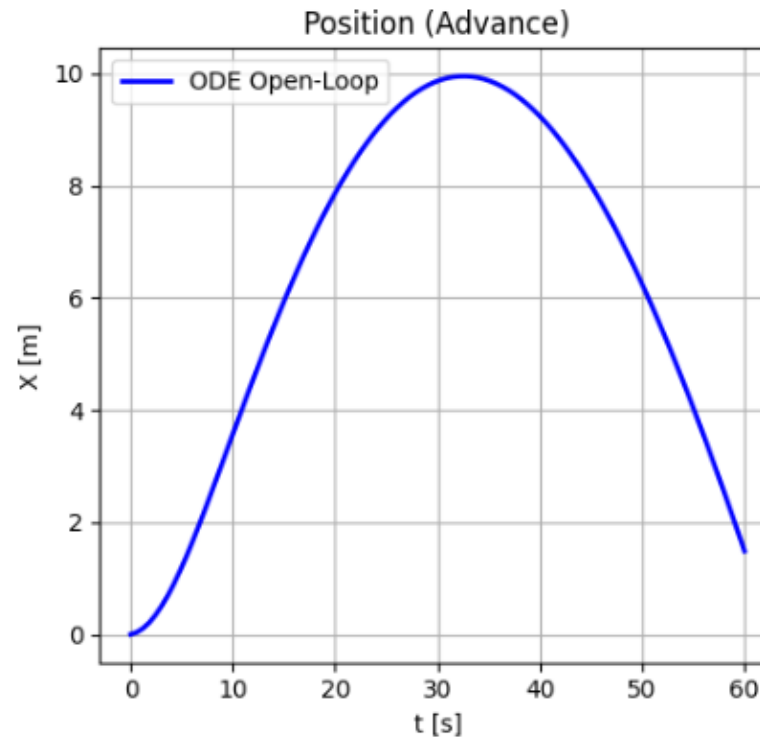
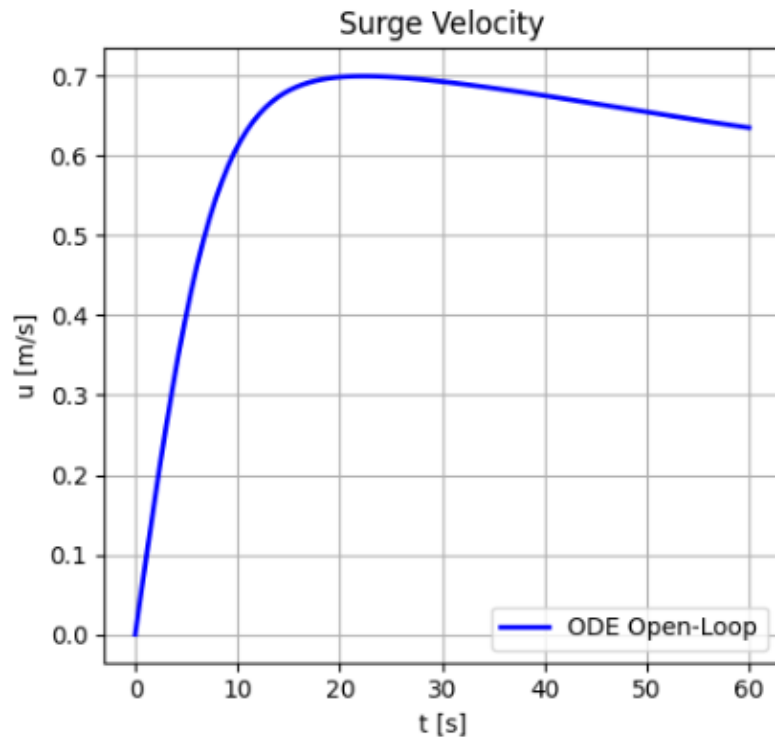
τ_{wave} : vector of wave-induced forces

$$\begin{bmatrix} m - X_{\dot{u}} & 0 & 0 \\ 0 & m - Y_{\dot{v}} & mx_g - Y_{\dot{r}} \\ 0 & mx_g - N_{\dot{v}} & I_z - N_{\dot{r}} \end{bmatrix} \begin{bmatrix} \dot{u} \\ \dot{v} \\ \dot{r} \end{bmatrix} + \begin{bmatrix} -X_u & 0 & 0 \\ 0 & -Y_v & (m - Y_{\dot{v}})U - Y_r \\ 0 & -N_v & (mx_g - Y_{\dot{r}})U - N_r \end{bmatrix} \begin{bmatrix} u \\ v \\ r \end{bmatrix} = \begin{bmatrix} \tau_1 \\ \tau_2 \\ \tau_6 \end{bmatrix}$$

ODE Simulation 1

Turning Maneuver

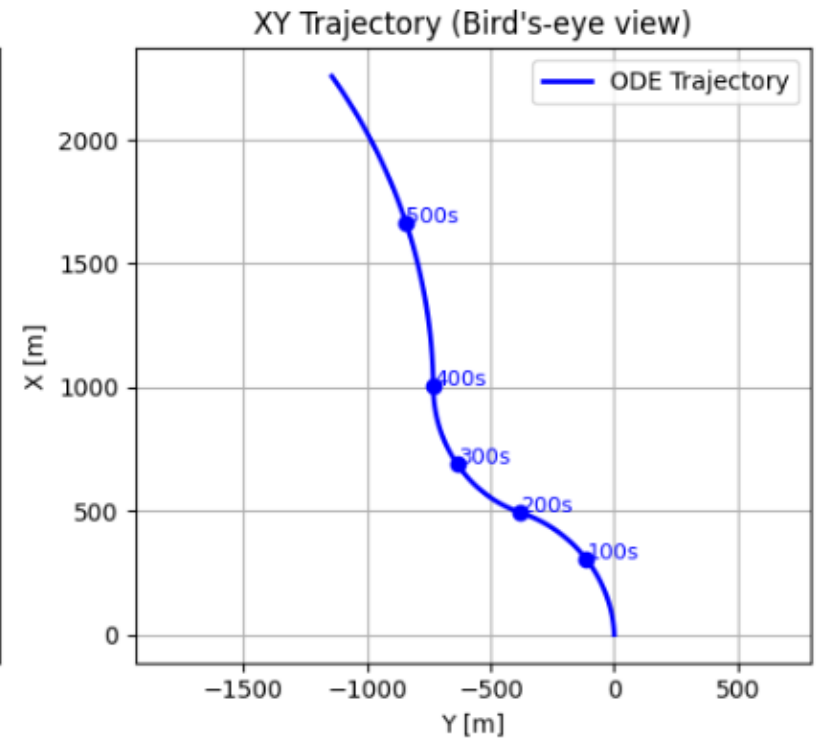
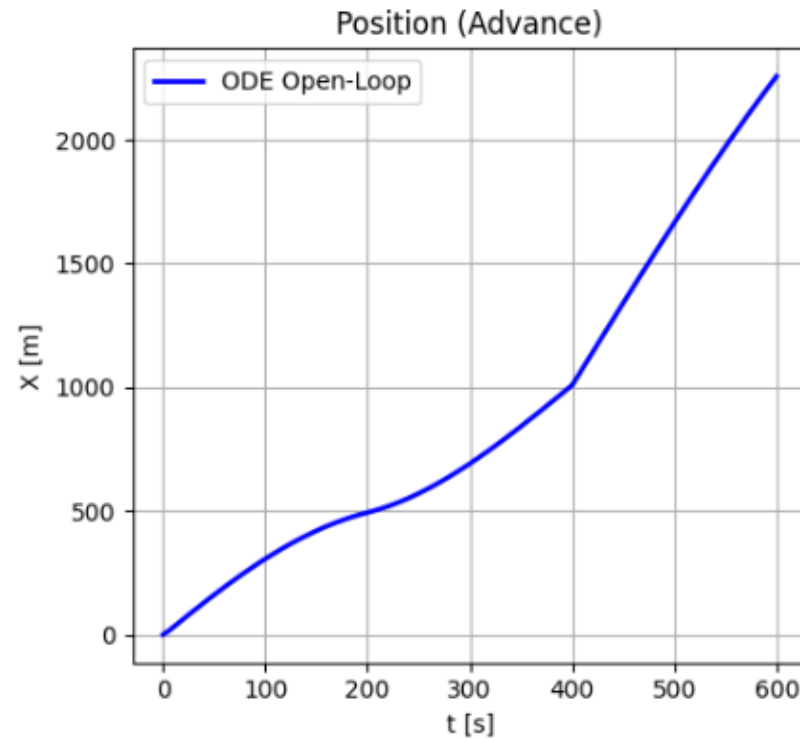
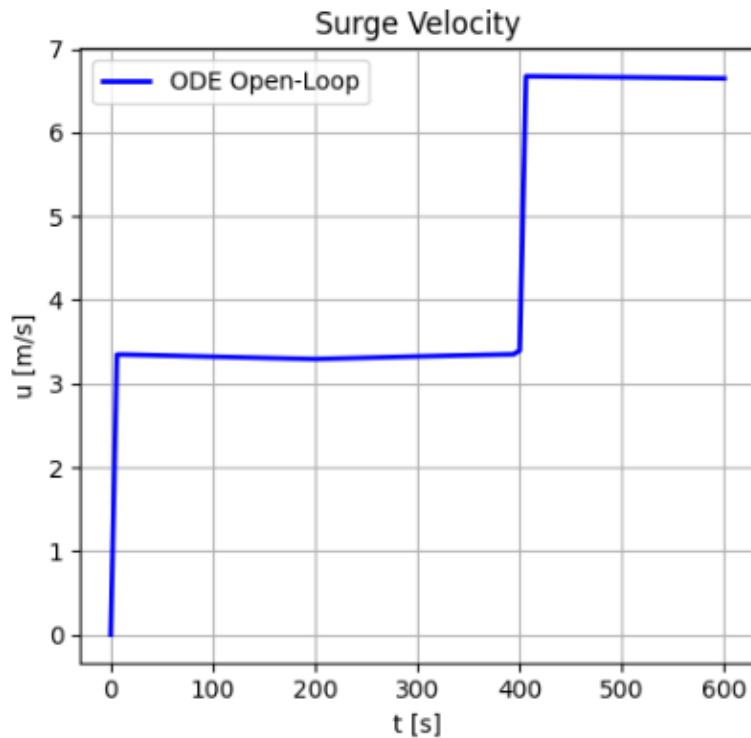
```
if t < 20:  
    n_eff = 2  
    delta_eff = np.deg2rad(-45)  
elif t < 40:  
    n_eff = 2  
    delta_eff = np.deg2rad(-45)  
else:  
    n_eff = 2  
    delta_eff = np.deg2rad(-45)
```



ODE Simulation 2

Zig-Zag Maneuver

```
if t < 200:
    n_eff = 10
    delta_eff = np.deg2rad(45)
elif t < 400:
    n_eff = 10
    delta_eff = np.deg2rad(-45)
else:
    n_eff = 20
    delta_eff = np.deg2rad(45)
```



ODE Simulation 3 – Environmental Impact

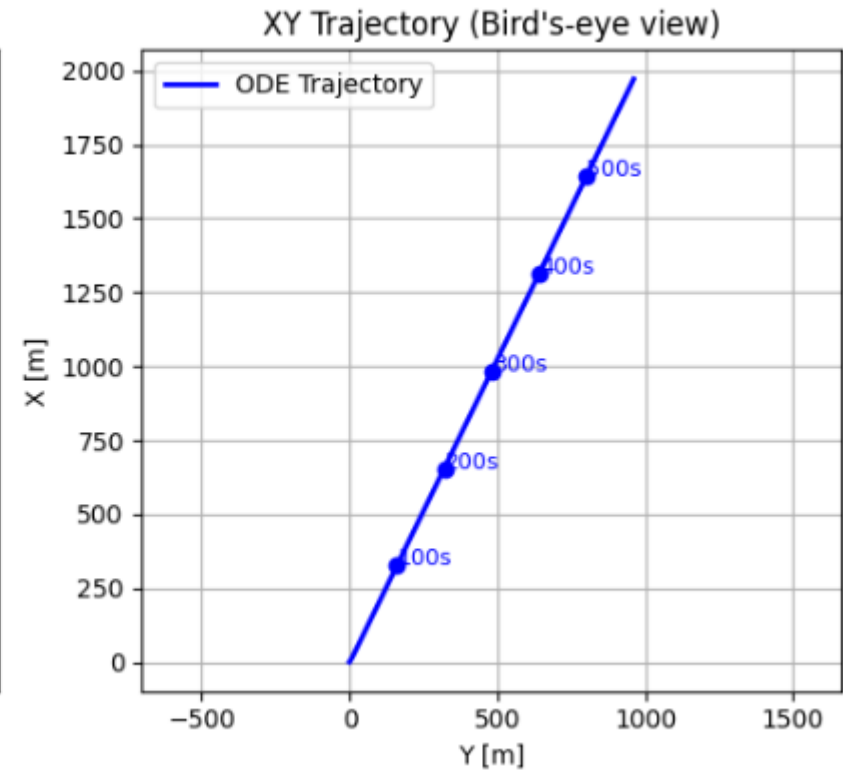
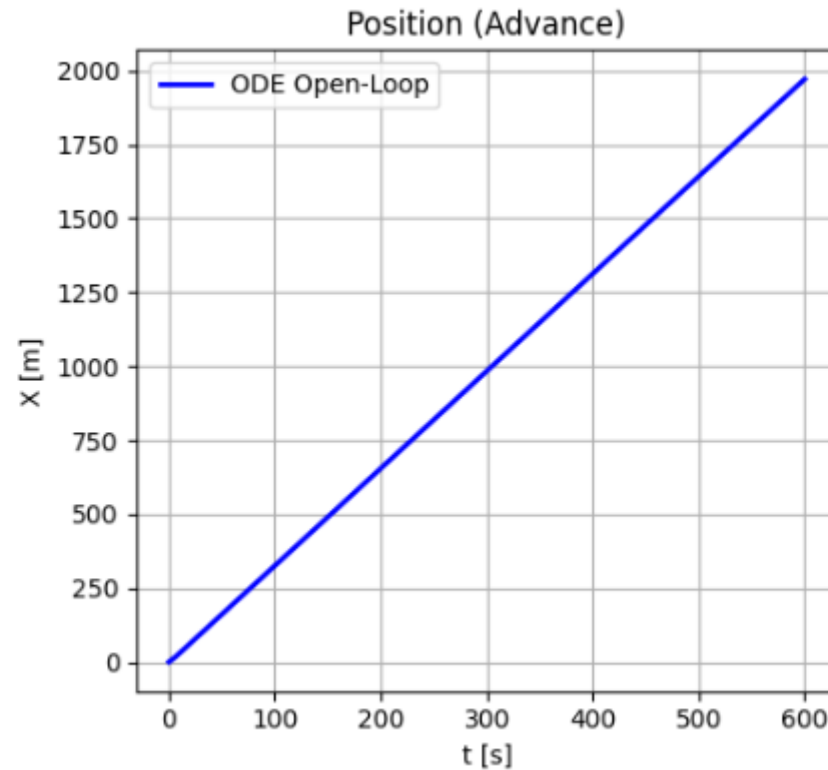
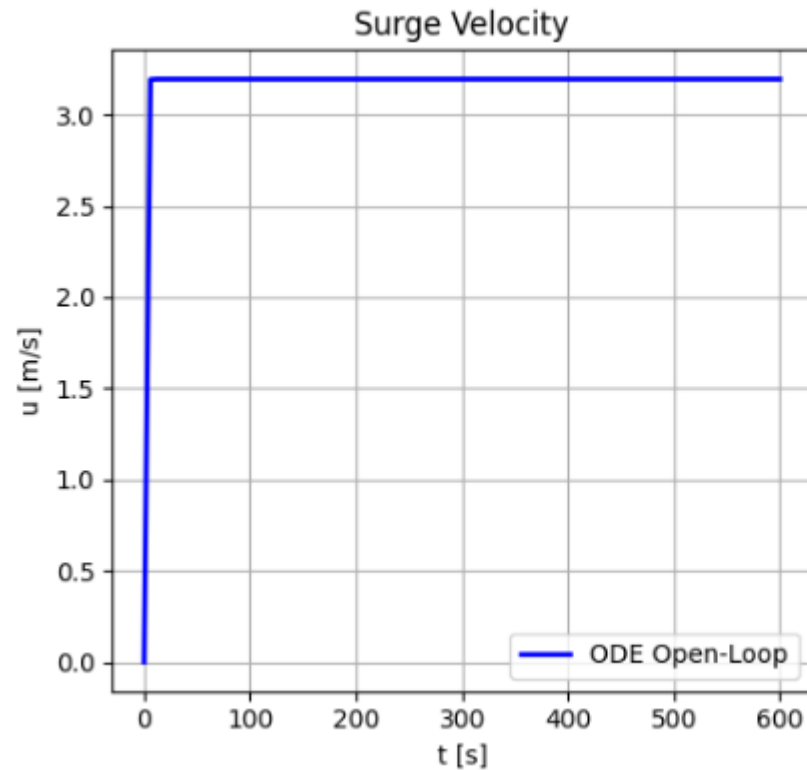
Steady Current

```
env = {  
    "current_speed": 1,  
    "current_dir_deg": 90,  
    "wind_force": 0.0,  
    "wind_dir_deg": 0.0,  
    "wind_yaw_arm": 0.0,  
    "current_speed_amp": 0,  
    "current_speed_freq": 0,  
    "wind_force_amp": 0.0,  
    "wind_force_freq": 0
```

```
if t < 200:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)  
elif t < 400:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)  
else:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)
```

ODE Simulation 3 – Environmental Impact

Steady Current



ODE Simulation 4 – Environmental Impact

Pulsating Current

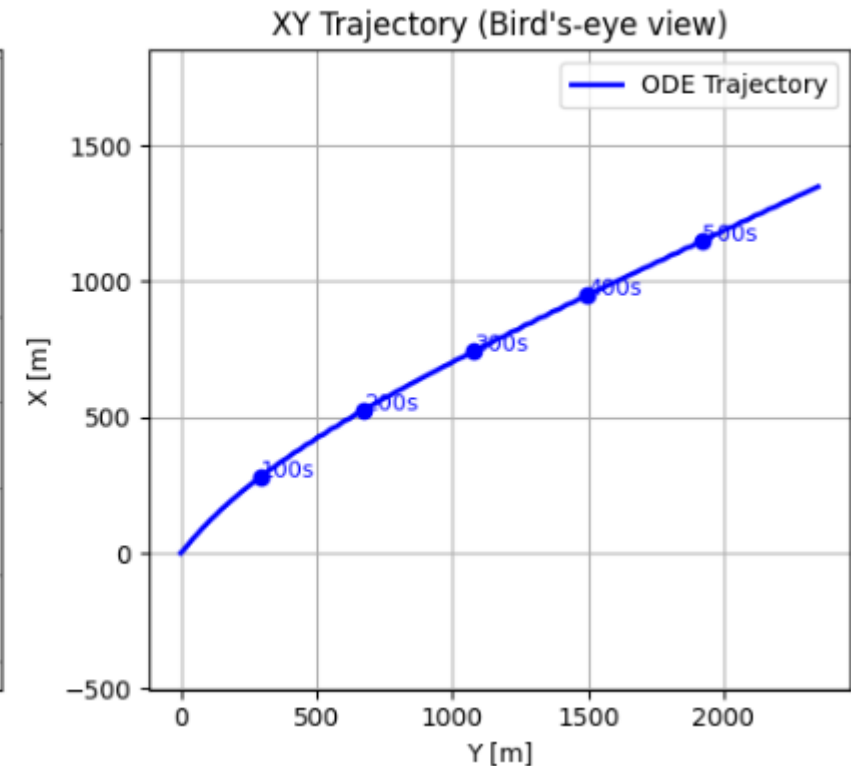
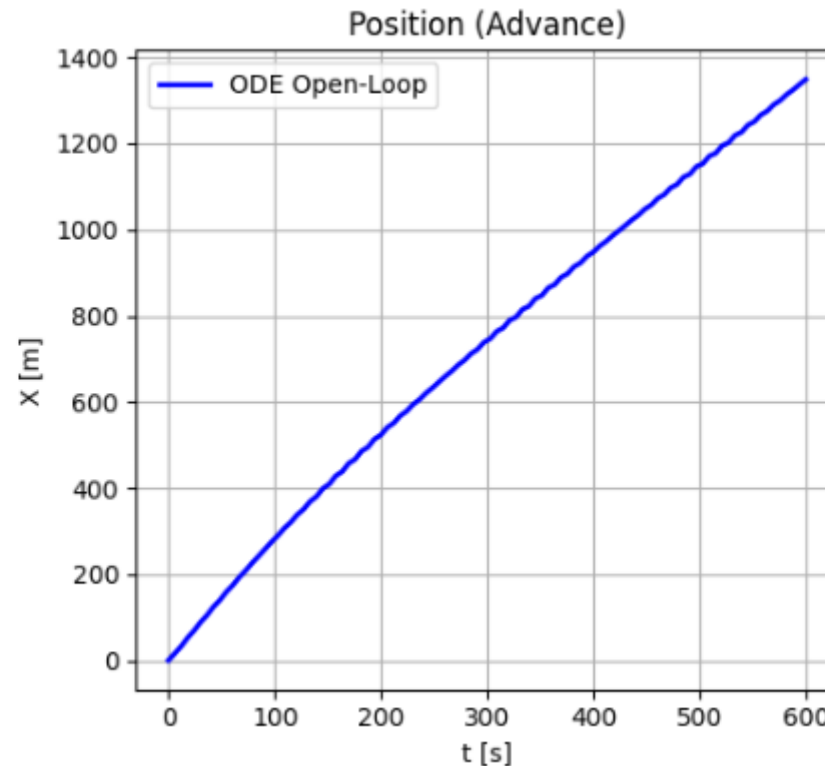
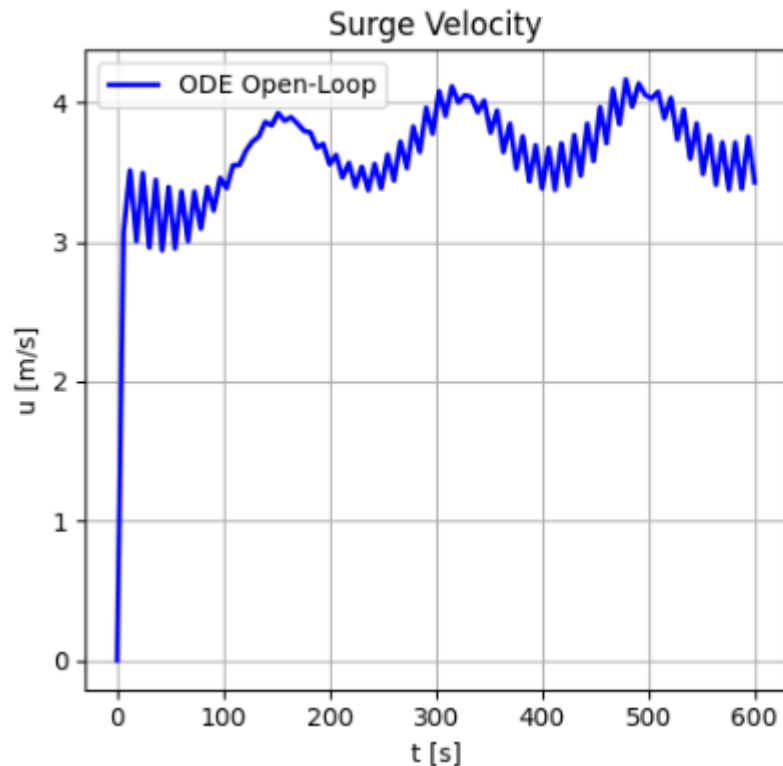
```
env = {  
    "current_speed": 1,  
    "current_dir_deg": 90,  
    "wind_force": 0.0,  
    "wind_dir_deg": 0.0,  
    "wind_yaw_arm": 0.0,  
    "current_speed_amp": 1,  
    "current_speed_freq": 0.5,  
    "wind_force_amp": 0.0,  
    "wind_force_freq": 0
```

```
def vec_from_mag_dir(magnitude, direction_deg):  
    angle = np.deg2rad(direction_deg)  
    return np.array([  
        magnitude * np.cos(angle),  
        magnitude * np.sin(angle)  
    ])
```

```
if t < 200:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)  
elif t < 400:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)  
else:  
    n_eff = 10  
    delta_eff = np.deg2rad(0)
```

ODE Simulation 4 – Environmental Impact

Pulsating Current



Linearized state-space matrices

A matrix (3 states x 3 states):

$\begin{bmatrix} -0.79430227 & 0. & 0. \end{bmatrix}$	➤ Surge
$\begin{bmatrix} 0. & -0.28268418 & -1.04286938 \end{bmatrix}$	➤ Sway
$\begin{bmatrix} 0. & 4.86497602 & -0.43161613 \end{bmatrix}$	➤ Yaw

B matrix (3 states x 2 inputs):

$\begin{bmatrix} 0. & 0.26377559 \end{bmatrix}$	➤ Surge
$\begin{bmatrix} -0.01082332 & 0. \end{bmatrix}$	➤ Sway
$\begin{bmatrix} -0.06058055 & 0. \end{bmatrix}$	➤ Yaw

Rudder angle Propeller speed

Linearization point:

- Surge: 2 m/s
- Sway: 0 m/s
- Yaw: 0 m/s

Linearized state-space matrices

A matrix (3 states x 3 states):

$\begin{bmatrix} -3.96185418 & 0. & 0. \end{bmatrix}$	➤ Surge
$\begin{bmatrix} 0. & -0.90532598 & -5.2519279 \end{bmatrix}$	➤ Sway
$\begin{bmatrix} 0. & 24.39102265 & -0.97954091 \end{bmatrix}$	➤ Yaw

B matrix (3 states x 2 inputs):

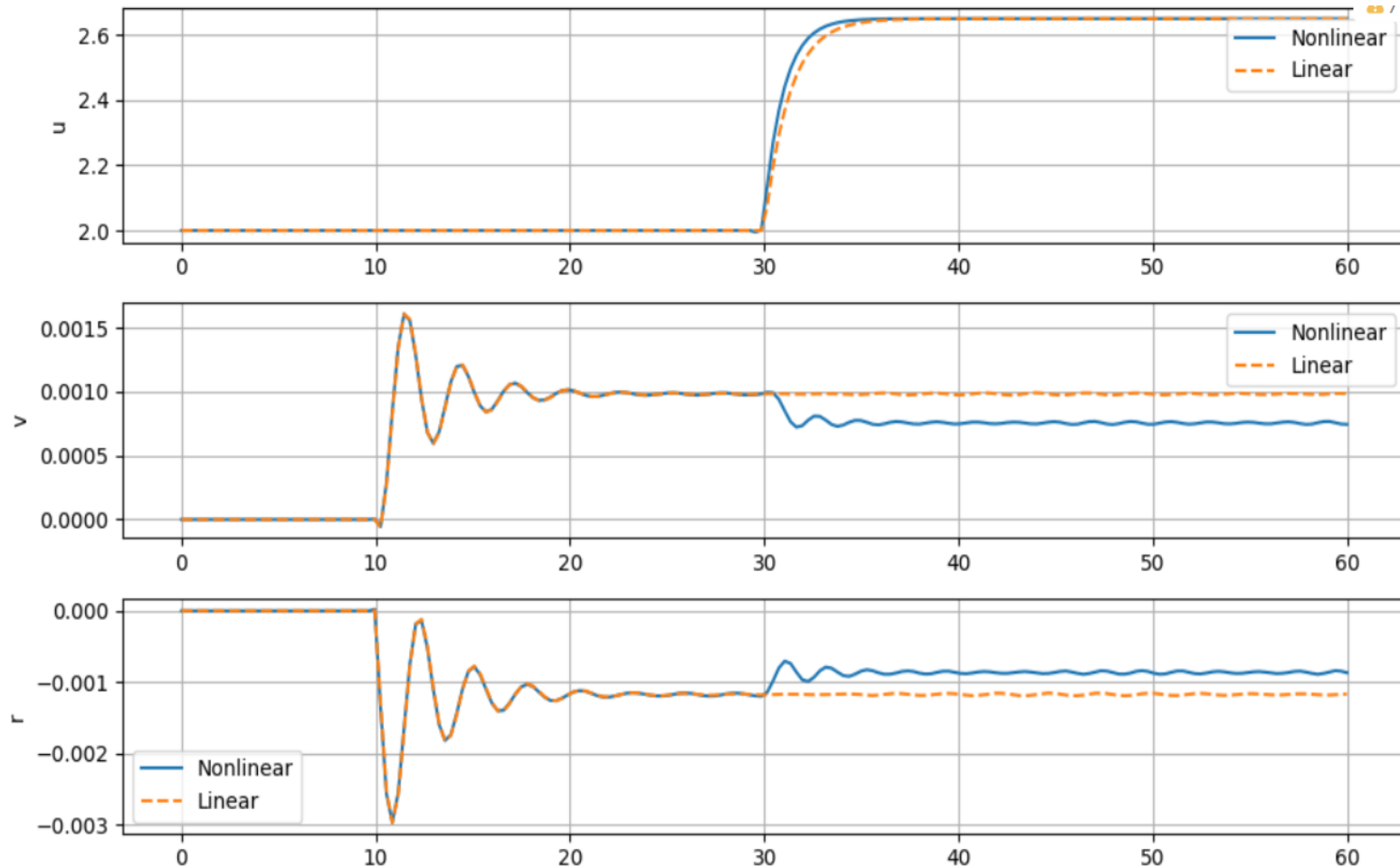
$\begin{bmatrix} 0. & 1.31567679 \end{bmatrix}$	➤ Surge
$\begin{bmatrix} -0.01082332 & 0. \end{bmatrix}$	➤ Sway
$\begin{bmatrix} -0.06058055 & 0. \end{bmatrix}$	➤ Yaw

Rudder angle Propeller speed

Linearization point:

- Surge: 10 m/s
- Sway: 0 m/s
- Yaw: 0 m/s

Comparison: nonlinear against linear



```
# Rudder step
if t < 10:
    delta = 0
elif t < 20:
    delta = np.deg2rad(5)
else:
    delta = np.deg2rad(5)
# Propeller speed variation
if t < 30:
    n = 6.0409
else:
    n = 8 # step up
return delta, n
```

Controllability and Observability

```
C_ctrl =  
[[1 0 0]  
 [0 0 1]]
```

```
D_ctrl =  
[[0 0]  
 [0 0]]
```

```
Co = cnt.ctrb(A, B)  
Obs = cnt.observ(A, C_ctrl)  
unco = len(A) - np.linalg.matrix_rank(Co)  
unob = len(A) - np.linalg.matrix_rank(Obs)
```

Uncontrollable modes: 0 | Unobservable modes: 0
The system is fully Controllable and Observable.

LQR

Boundaries

- *max allowable surge velocity error [m/s]* = 0,5
- *max allowable sway velocity [m/s]* = 0,3
- *max allowable yaw-rate error [°/s]* = 10
- *rudder limit [°]* = 35
- *max allowable propeller deviation [rps]* = 5

```
u_dev_max = 0.50
v_max = 0.3
r_max = np.deg2rad(10.0)

delta_max = np.deg2rad(35.0)
dn_max = 5.0

x_max = np.array([u_dev_max, v_max, r_max])
u_max = np.array([delta_max, dn_max])

q_priority = np.array([1.0, 1.0, 100.0])
r_priority = np.array([1.0, 1.0])
```


MPC

Limits

Rudder minimum limit: -35 deg

Rudder maximum limit: +35 deg

Propeller maximum limit: 10 rps

Propeller minimum limit: prevents the n from becoming null or negative

```
umin_mpc = np.array([
    -np.deg2rad(35.0),
    -float(n0_val) + 0.1
])

umax_mpc = np.array([
    np.deg2rad(35.0),
    10.0
])
```

PID

Transfer Functions

$$\frac{u(s)}{\delta(s)} = 0$$

$$\frac{u(s)}{n(s)} = \frac{0.264 s^2 + 0.188 s + 1.37}{s^3 + 1.509 s^2 + 5.763 s + 4.127}$$

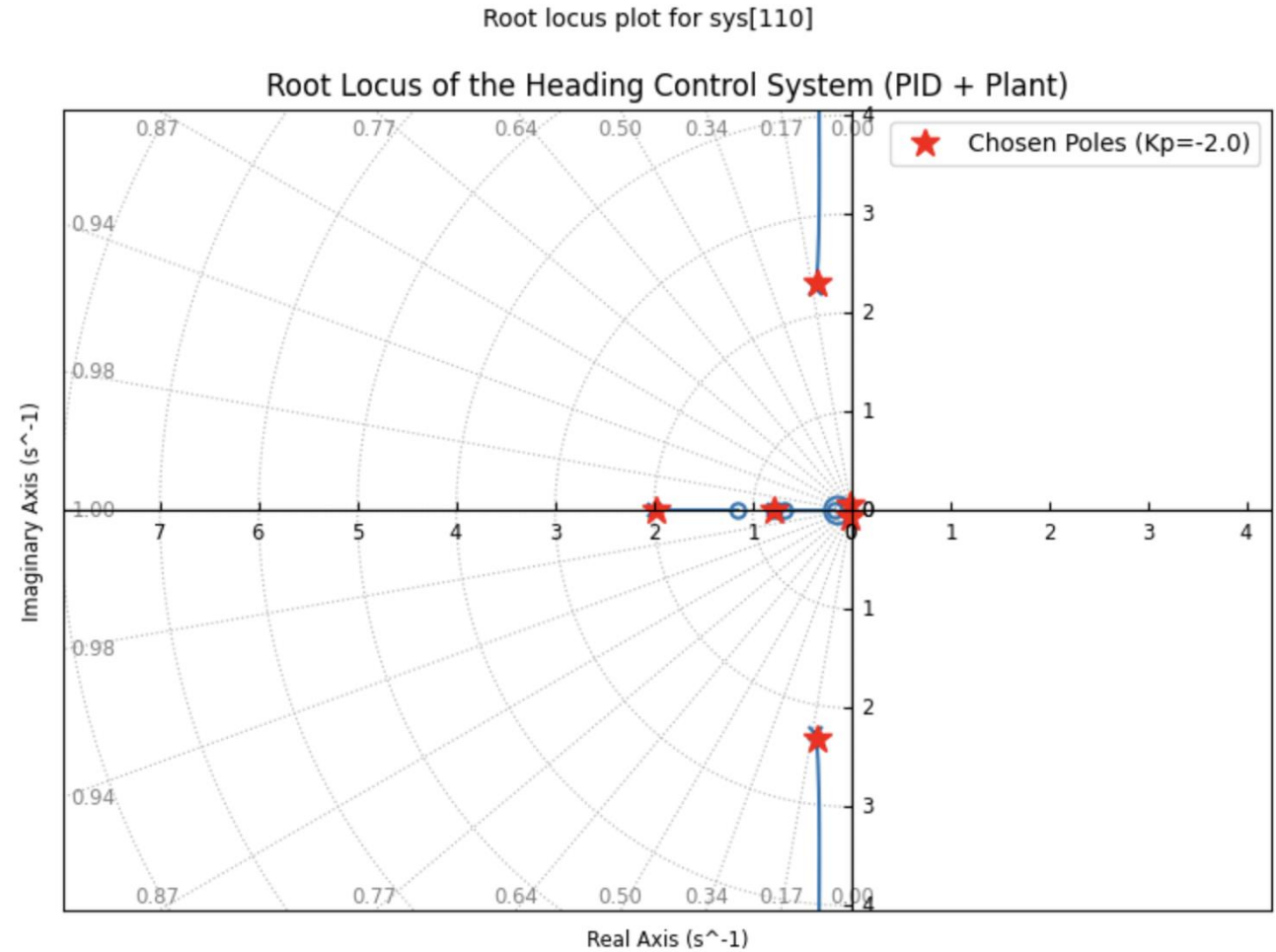
$$\frac{r(s)}{\delta(s)} = \frac{-0.061 s^2 - 0.118 s - 0.055}{s^3 + 1.509 s^2 + 5.763 s + 4.127}$$

$$\frac{r(s)}{n(s)} = 0$$

PID

Root Locus

- Blue trajectories: migration of the system's six poles as the proportional gain varies
- Red stars: our selected operating point ($K_p = -2.0$)



Simulation

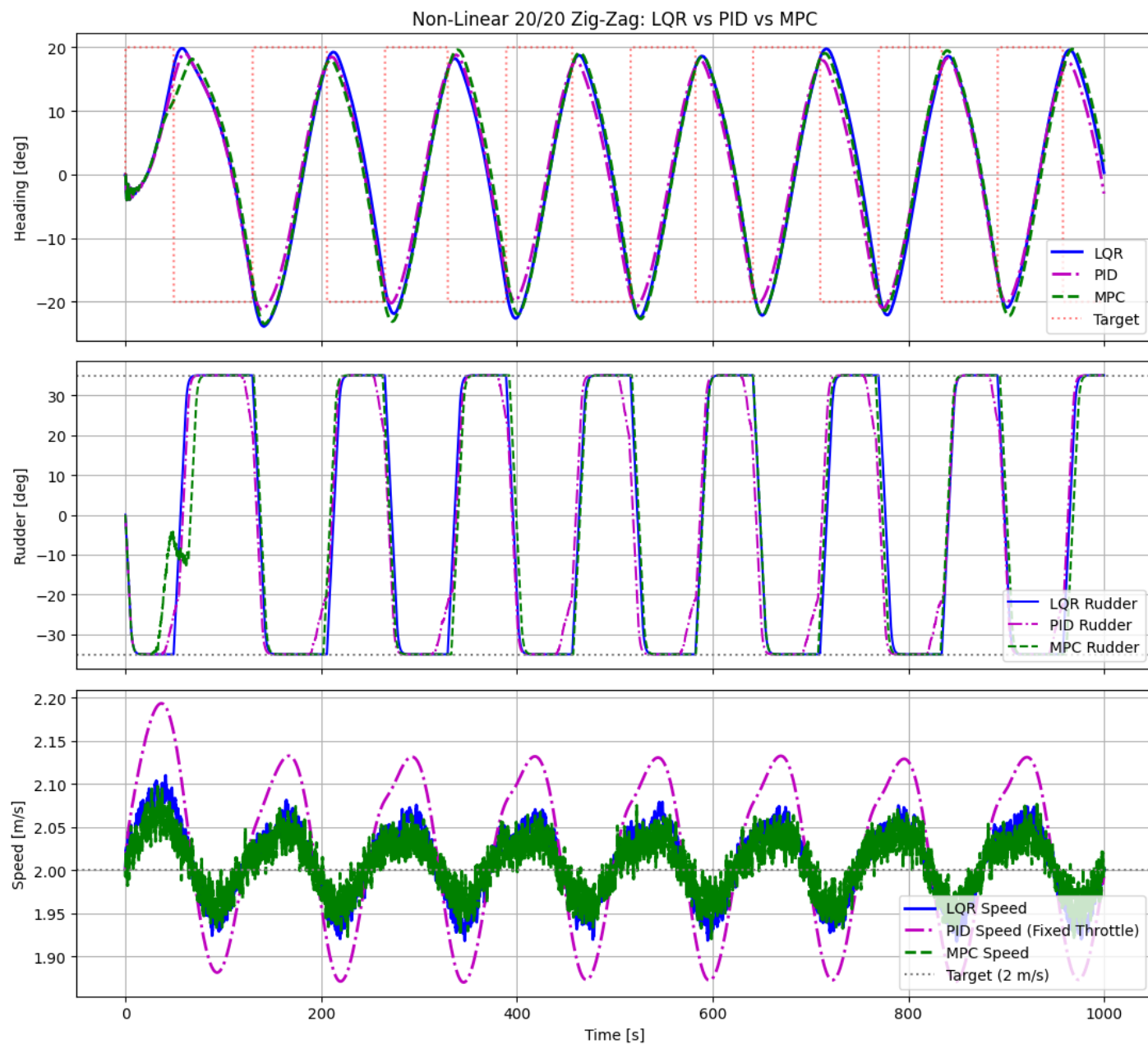
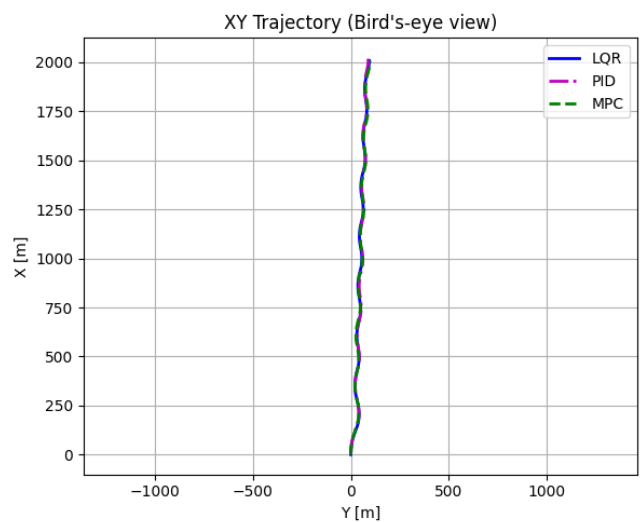
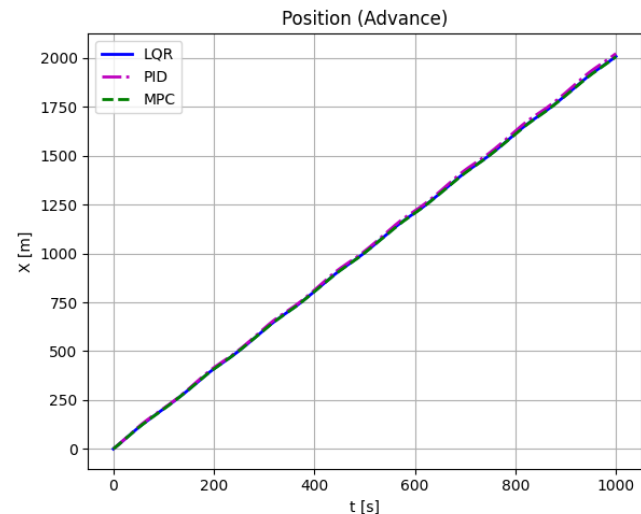
- Slowly varying ocean current to induce a persistent lateral drift
- Gaussian white noise on the state measurements to simulate the inherent inaccuracies of standard real-world navigational sensors
- Simulation time: 1000 s

```
# Realistic Disturbances
env["current_speed"] = 0.05
env["current_dir_deg"] = 45.0
env["wind_force"] = 0.1
env["wind_dir_deg"] = -30.0
```

```
# Noise
noise_std_u = 0.05
noise_std_r = np.deg2rad(0.5)
noise_std_psi = np.deg2rad(1.0)
```

```
# Setup
dt = 0.1
time_sim = np.arange(0, 1000, dt)
n_steps = len(time_sim)
```

Results



Control Strategy Optimization Metrics

Metric	LQR	PID	MPC
IAE [deg*s]	26131.67	24353.12	25658.53
Control Effort [deg*s]	34995.40	33035.28	34436.21

PINN Simulation

- Correct dynamic trends
- Limited local accuracy
- Magnified deviations

